ChatGPT

plans-app 无服务器改进研究报告

执行摘要

基于 GitHub [1] 连接器对 `Distance0x/plans-app` 的静态审查，我的判断是：这个项目已经不是“从 0 到 1”，而是“从可用原型到稳定单机产品”的阶段。README 和仓库文档显示，它已经具备本地优先的任务管理、清单/标签、日历、提醒、重复、番茄钟、浮窗、附件、导入导出等基础能力，而且明确定位为 **Windows 单机本地应用**，当前 **不做自动多机同步**。 [2] [3]

你的核心约束——**没有服务器、不能做多端协同**——并不会阻止这款产品继续变强；它真正限制的是“账号/同步/协作”，而不是“智能计划”。相反，这个约束非常适合把产品做成“单机个人规划中枢”：本地数据库负责事实存储，AI 只负责解析、拆解、排序和排程建议，最终动作都在本地执行。官方文档也说明，OpenAI [4] 的 Responses API 本身就支持多轮对话、函数调用和结构化输出，因此完全可以在 **无服务器架构** 下做一个“对话式排程 agent”。 [5]

我对当前版本的总体结论是：**最优先的不是加更多页面，而是先修“可靠性和入口收口”**。当前最值得先补的短板是：数据库落盘策略、备份一致性、时间/时区处理、提醒生命周期、番茄钟完成路径、搜索/过滤性能、功能入口不一致，以及测试/类型/文档漂移。修完这些之后，再上 AI agent、自然语言快速添加、自动排程、ICS 导出，会比现在直接堆功能更稳。 [6] [7] [8] [9] [10]

结论先说清楚：**可以和 AI 对话，让它帮你创建任务和规划日程，而且这是我最推荐你优先做的功能之一**。但实现方式不应该是“把 API key 放到前端 localStorage 然后直接 fetch”，而应该是 **Electron 主进程代理调用 + 本地工具函数 + 草稿确认后写入**。这能兼顾你的无服务器前提、可用性和安全性。Electron 官方推荐开启 context isolation、按消息粒度暴露安全 API；Electron 还提供 `safeStorage` 作为本地机密存储能力。 [11]

当前实现与约束

当前代码结构已经形成了比较标准的 Electron 双进程架构：主进程负责窗口、IPC、sqlite/sql.js、提醒、通知、托盘；渲染进程是 React + Zustand；preload 通过 `contextBridge` 暴露 API。`BrowserWindow` 已启用 `contextIsolation: true` 且关闭 `nodeIntegration`，这是一个不错的安全基线。数据库使用 `sql.js + Drizzle`，数据落在用户本地目录。 [12] [13] [6] [14]

```
flowchart LR
    U[用户] --> R[React Renderer]
    R --> P[Preload IPC Bridge]
    P --> M[Electron Main]
    M --> DB[(Local SQLite/sql.js)]
    M --> N[Notification / Reminder / Tray]
    M --> FS[Attachments / Backup / ICS]
    M --> AI[AI API]
```

你给出的明确约束与我据此采用的假设如下：

项目	结论
服务器	明确没有
多端协同	明确不能做
目标平台	**未指定**；现仓库实现为 Windows 桌面 Electron
AI 提供商	**未指定**；下文用 OpenAI 官方 API 作为具体示例
外部日历类型	**未指定**；建议先做通用 ICS，而不是 OAuth
Web/PWA	**未指定**；若未来要做，只建议作为 companion，不建议替代当前桌面主线

README 已明确写出：当前数据是本地存储，不依赖服务器，不同机器之间不会自动同步；仓库自己的迭代文档也把账号、同步、协作、开放 API 后置。换句话说，“无服务器单机版”不是推测，而是当前产品边界。 [2] [3]

如果未来你想补一个 Web/PWA 版本，也不是不能做，但我不建议把它作为当前第一优先级。原因是：浏览器侧的 Background Sync 不是 Baseline，Notifications 和 File System / Access API 都依赖 secure context，且不同浏览器支持差异明显；这些能力在 Electron 桌面端反而更稳定。 [15]

主要不足与缺陷

数据可靠性不足。 `electron/database/db.ts` 第 11–41 行在首次初始化后启动了一个永久 `setInterval`，每 5 秒将整个 `sql.js` 数据库 `export()` 后 `fs.writeFileSync()` 落盘。这种策略有三个问题：一是 **崩溃时存在最多约 5 秒的数据丢失窗口**；二是 **主进程同步写磁盘**，会把 I/O 抢占放到主线程；三是它不是基于事务或写后立即持久化的策略。对个人规划工具来说，这比“同步缺失”更先影响信任感。 [6]

导入/恢复缺少原子性，而且附件不会跟着走。 `electron/ipc/backup-handler.ts` 第 41–82 行导入时先删多张表再逐张插回，没有事务、没有回滚；如果中途失败，数据库会停在半恢复状态。与此同时，第 10–38 行导出的只是表数据，第 85–107 行附件选择却把文件复制到 `userData/attachments`，这意味着 **JSON 备份并不包含附件实体文件**，跨机器手动恢复后会出现“任务在、附件路径失效”的情况。 [7]

时间/时区处理不一致。 `src/App.tsx` 第 205–219 行用 `new Date().toISOString().split('T')[0]` 计算“今天”，同时又把 `YYYY-MM-DD` 直接喂给 `new Date(t.dueDate)` 做近期比较；而日历和浮窗又使用本地格式化函数。这种“UTC 字符串 / 本地日期 / 原生 Date 解析”混用，很容易在东八区晚间、跨午夜和 DST 边界出现“今天任务跑到昨天/明天”的问题。 [9] [16] [17]

番茄钟存在完成路径竞态，而且没有恢复机制。 `electron/ipc/timer-handler.ts` 第 147–159 行在倒计时结束后直接调用 `completeSession(); switchToNextSession();`，但这里没有等待 `completeSession()` 完成；而 `completeSession()` 内部会异步更新数据库并增加 `completedPomodoros`。这意味着“是否进入长休息”的判断有机会用到旧计数，属于明显的竞态风险。更重要的是，当前计时状态主要存在内存变量 `timerState` 中，应用重启或系统睡眠后没有恢复逻辑，而 Electron 官方提供了 `powerMonitor.resume` 正是为这类修正场景准备的。 [8] [18]

提醒的可靠性依赖“应用必须活着”。 `electron/services/reminder-engine.ts` 采用 15 秒轮询 pending reminders；`electron/main/index.ts` 第 180–185 行又在所有窗口关闭时停止提醒引擎并退出应用。也就是说，现在的提醒严格依赖应用仍在运行。结合 `electron/utils/tray.ts` 的逻辑可见：最小化会隐藏到托盘，但点击关闭则会真正退出。对计划类工具来说，这会让用户误以为“我关了窗口但提醒应该还在”，实际却不是。 [19] [12] [20]

搜索和过滤路径重复，且大部分筛选还在内存里做。 `TaskFilters` 第 31–36 行在每次输入时同时触发 `onSearch` 和 `onFilterChange`；`TaskList` 第 28–31 行挂载时又会主动 `fetchTasks()`。更深一层，`electron/ipc/task-handler.ts` 第 232–305 行虽然先查数据库，但标签筛选、搜索词聚合、Reminder/Recurring 判断大量在 JS 内存中二次处理。这会造成 **一次输入触发多次查询/刷新**，并且随着任务量增长越来越慢。 [10] [21] [22]

功能入口不一致，已经写好的能力没有真正形成主链路。 `QuickAddTask` 的提交 payload 只覆盖标题、时间、时长、优先级、附件；日期面板里虽然渲染了“提醒”“重复”按钮，但没有真正接入状态和提交流程。相反，`TaskForm.tsx` 已经实现了提醒、重复规则、标签、附件等完整表单，但按仓库搜索，`TaskForm` 只命中组件自身和文档，没有命中明确的主流程调用点，因此我判断它**大概率尚未接入当前主界面**。这不是“没有功能”，而是“功能写了但用户走不到”。 [23] [24] [25] [26]

子任务能力与文档预期有落差。 `src/components/tasks/TaskItem.tsx` 第 24–25 行把嵌套条件写成了 `level < 2`，也就是层级上限被硬编码；而实现计划文档里曾把子任务能力描述为“支持多层嵌套”。如果这是产品决策，应该显式写约束；如果不是，就是隐藏限制。 [27] [28]

类型、安全边界和工程基线还不够产品化。 preload 虽然用了 `contextBridge`，但仍暴露了任意频道的 `on/off` 包装，没有 allowlist 或 schema 校验；`src/types/electron.d.ts` 和多个 store / 组件中充满 `any`；`package.json` 也没有 test / lint / format 脚本，而架构文档和实现计划却多次提到 ESLint、Prettier、单元/集成/E2E 测试。Electron 官方建议按消息粒度暴露 IPC，并强调不要把过于宽泛的能力直接给渲染进程；从当前工程状态看，你已经有安全基线，但还没有产品级的“类型与验证基线”。 [13] [29] [30] [31] [28] [14]

文档与实现存在明显漂移。 `docs/02-技术架构设计.md` 里的目录仍然列着 `ai-handler.ts`、`updater.ts`、`tests/`、`electron-builder.json` 等规划项，但当前真实代码路径已经演化为 `electron/main/index.ts`、`electron/preload/index.ts` 等结构，而且并没有对应的自动更新和测试落地。`src/main.tsx` 里也保留了 `TestApp` 相关调试开关。长期看，这会让你以后引入 AI、备份升级、插件化时付出更高的认知成本。 [31] [32] [30]

可行功能与 AI agent 设计

对照官方产品资料，滴答清单的高频价值集中在 **自然语言录入、重复提醒、过滤器和多视图日历**；Sunsama 与 Akiflow 的差异化则集中在 **timeboxing、自动排程和 AI 助手**。这说明你在单机无服务器场景下最该优先借鉴的，并不是协作/同步，而是“快速录入 + 时间块 + 自动排程 + 可靠提醒 + 对话式助理”。 [33]

Feature	Offline-capable (Y/N)	Complexity	Priority	Estimated days
AI 对话式计划 Agent	N	高	P0	5–7
自然语言快速添加	Y	中	P0	2–3

Feature	Offline-capable (Y/N)	Complexity	Priority	Estimated days
本地自动排程与空闲时段推荐	Y	中	P0	3–4
托盘后台运行、开机启动与提醒恢复	Y	中	P0	2–3
手动同步包（备份/恢复/附件/回滚）	Y	中	P0	3–4
ICS 日历导出/导入	Y	中	P1	2–3

AI 对话式计划 Agent。

- **描述与用户流程：** 用户说“把这周没完成的任务重排到明晚前”或“帮我把 PRD 拆成 5 个任务并安排到本周”，agent 先读取本地任务、工作时间、已有时间块，产出 **草稿计划**，用户确认后再写入数据库。写操作一律采用 “draft → confirm → apply → undo” 模式。Responses API 支持多轮状态、函数调用和结构化输出，正适合这种模式。 [34]
- **代码改动：** 新增 `electron/ipc/ai-handler.ts`、`electron/services/ai-service.ts`、`electron/services/keyvault.ts`、`electron/services/scheduling-engine.ts`、`src/components/ai/AgentPanel.tsx`、`src/stores/agent-store.ts`；修改 `electron/preload/index.ts`、`src/types/electron.d.ts`、`src/App.tsx`。
- **复杂度 / 优先级：** 高 / P0。它最贴合你的“无服务器 + 有 API key”条件，而且能直接提升产品差异化。

自然语言快速添加。

- **描述与用户流程：** 在现有 Quick Add 输入框里直接输入“明天下午 3 点给小李发报价，30 分钟，高优先级”，本地规则引擎先做 80% 解析；解析失败时再调用 AI 兜底。这样大部分常见输入可以离线工作，复杂表达再在线增强。
- **代码改动：** 修改 `src/components/tasks/QuickAddTask.tsx`；新增 `src/lib/nlp/parse-cn-time.ts`、`electron/services/nlp-fallback.ts`；复用 `task-handler.ts` 的现有写入逻辑。现有 QuickAdd 与完整 TaskForm 能力脱节，这个改造正好把入口统一起来。 [23] [24]
- **复杂度 / 优先级：** 中 / P0。它是 AI agent 的轻量前置版本，投入小、收益大。

本地自动排程与空闲时段推荐。

- **描述与用户流程：** 用户点“规划今天”后，系统根据已有任务、截止时间、优先级、时长、工作时段，先本地算出 free slots，再给出建议时间块；AI 只负责解释和调优权重，不直接替代排程引擎。
- **代码改动：** 新增 `electron/services/scheduling-engine.ts`、`electron/services/date-utils.ts`；修改 `src/components/calendar/Calendar.tsx`、`src/stores/task-store.ts`、`src/stores/settings-store.ts`。Sunsama 与 Akiflow 的 timeboxing 能力证明，这一块是真正能拉开工具价值的区域。 [35]
- **复杂度 / 优先级：** 中 / P0。建议采用“确定性引擎 + AI 解释”的组合，而不是纯模型排程。

托盘后台运行、开机启动与提醒恢复。

- **描述与用户流程：** 关闭主窗口默认不退出，只最小化到托盘；开机可自启动；系统睡眠恢复后自动校正番茄钟与提醒状态。这样你虽然没有服务器，但单机上的“持续存在感”会强很多。Electron 官方提供了托盘、登录启动和 `powerMonitor.resume` 能力，完全符合无服务器方案。 [36]

- **代码改动：** 修改 `electron/main/index.ts`、`electron/utils/tray.ts`、`electron/ipc/settings-handler.ts`、`src/stores/settings-store.ts`；新增 `electron/services/session-recovery.ts`。
- **复杂度 / 优先级：** 中 / P0。它优先级高于“更多炫酷 UI”，因为它决定提醒到底靠不靠谱。

手动同步包。

- **描述与用户流程：** 不做云同步，但做一个 `plans-bundle.zip`：里面包含 JSON 数据、附件目录、版本号、校验信息、导入预检结果和失败回滚点。用户可以手工在两台机器之间迁移，这就是“无服务器版的手动 sync”。
- **代码改动：** 修改 `electron/ipc/backup-handler.ts`；新增 `electron/services/export-bundle.ts`、`electron/services/import-validator.ts`；数据库新增 `attachments` 表和 `snapshots` / `restore_points` 表。当前备份只导出表数据、不带附件，且导入不回滚，这一项会直接补上当前最大的数据完整性短板。[7]
- **复杂度 / 优先级：** 中 / P0。它不是“多端协同”，但它是无服务器前提下最现实的数据迁移方案。

ICS 日历导出/导入。

- **描述与用户流程：** 用户选中某天/某个清单/某个过滤器，导出 `.ics` 文件并一键调用系统默认日历打开；也可以从本地导入 `.ics` 把外部安排落回 plans-app。iCalendar 是通用的日历交换标准，这条路比 OAuth 更适合你的当前阶段。[37]
- **代码改动：** 新增 `electron/services/ics-service.ts`、`electron/ipc/calendar-export-handler.ts`；修改 `src/components/calendar/Calendar.tsx`、`src/App.tsx`、`electron/preload/index.ts`。
- **复杂度 / 优先级：** 中 / P1。建议先做单向导出，再做受控导入，不要第一期就碰 Google/Outlook OAuth。

关于 PWA 的取舍。
我建议把它放到 **P2 或更后**。如果以后你想要“随手打开的轻 Web 版本”，可以做一个只支持查看、快速添加和本地文件导入导出的 companion PWA；但在当前阶段，不应该让 PWA 改变桌面主架构。原因很简单：浏览器侧后台同步和通知能力不如 Electron 稳，且安全上下文要求更多。[38]

AI agent 的实现结论。
我建议的架构是：**不要让 Renderer 直接拿 API key 请求模型；把所有模型请求都收敛到主进程。**

方案	是否无服务器	Key 暴露面	实现复杂度	推荐度
Renderer 直接 fetch	是	高	低	低
Main process 代理 + preload 最小接口	是	中低	中	**高**
本地 sidecar 进程	是	低	高	中

Responses API 可以用 `previous_response_id` 维持连续对话，Structured Outputs 能让模型稳定返回 JSON，Function Calling 则适合把“创建任务 / 更新任务 / 查空闲时段 / 应用排程”定义成工具。要注意的是：官方文档明确说明，即使用 `previous_response_id`，前面对话的输入 token 仍会按输入计费；速率限制则按组织/项目、模型、RPM/TPM/RPD/TPD 和 usage limit 共同控制。[39]

```
flowchart TD
    A[用户输入] --> B[AgentPanel]
    B --> C[Preload: ai.chat]
    C --> D[Main: ai-service]
    D --> E[读取本地上下文 tasks/settings/free slots]
    E --> F[Responses API]
    F --> G{Tool calls?}
    G -- 是 --> H[本地工具: create/update/find_free_slots/schedule]
    H --> I[返回 tool result]
    I --> F
    G -- 否 --> J[生成草稿]
    J --> K[用户确认]
    K --> L[写入 SQLite]
    L --> M[通知/日历/提醒刷新]
```

安全与隐私。
桌面端最好的本地机密存储方案不是自己造轮子，而是 Electron 的 `safeStorage`。它会使用操作系统提供的加密能力；但文档也明确提醒，Linux 上如果没有可用 secret store，后端可能退化到 `basic_text`。因此我建议：Windows/macOS 默认 `safeStorage`；Linux 检测到 `basic_text` 时给用户两个选项——“仅本次会话保存”或“主密码加密”。同时，**不要把 API key 放进普通 settings 表，也不要默认打包进备份文件**。 [40]

成本与限流。
按 2026-04-28 可见的官方 API Pricing 页面，`GPT-5.4 mini` 价格是输入 `$0.75 / 1M tokens`、输出 `$4.50 / 1M tokens`。按这个价格粗估：

- 一次“快速解析成任务”（约 600 input / 250 output）成本约 **$0.0016**。
- 一次“当天排程”（约 2500 input / 900 output）成本约 **$0.0059**。
- 一次“周复盘/重排”（约 8000 input / 2000 output）成本约 **$0.015**。
  如果你把“解析”与“排程解释”都放在 mini 模型上，个人使用的月成本通常是可控的；真正要防的不是单次费用，而是 **无节制上下文累积** 与 **错误重试风暴**。 [41]

分阶段改进计划

我建议按 **先修基线、再上 agent** 的顺序推进，而不是反过来。仓库自己的文档把 AI/NLP 放在偏后位置，但在你“已有 API key、又不做同步”的前提下，我会把 AI 提前到 P0/P1，因为它比账号体系和云协同更符合当前产品边界。 [3]

Milestone	估时	交付物	测试点
基线修复	3 天	修复 SQL 落盘、时间/时区、番茄钟竞态、关闭/托盘语义、删掉 TestApp 调试遗留	数据不丢、今天/近期视图正确、番茄钟切段正确
可靠提醒层	3 天	开机启动、后台常驻、恢复校正、提醒生命周期重构	睡眠恢复、关主窗不丢提醒、重复任务提醒延续

Milestone	估时	交付物	测试点
本地排程层	4 天	空闲时段引擎、QuickAdd 中文解析、草稿式排程 UI	来自自然语言的任务字段准确、排程不覆盖固定块
AI agent 层	5–6 天	key 管理、AI IPC、聊天面板、工具调用、确认/撤销	不经确认不写库、工具参数校验、429/backoff、上下文压缩
数据迁移与日历层	4 天	bundle 备份、附件随包、导入预检/回滚、ICS 导出/导入	异机恢复成功、附件可打开、ICS 可被系统日历识别

推荐总工期：19–20 天。 如果时间紧，只保留到 “AI agent + 基线修复 + 手动同步包”，去掉 ICS，也可以在约 15–17 天内交付一个很强的单机版本。

风险	影响	缓解
API key 泄露	高	主进程代理调用；`safeStorage`；Linux 弱后端时启用 session-only / 主密码模式
AI 幻觉导致误写任务	高	所有写操作先生成草稿；强制确认；保留 `plan_snapshots` 支持一键撤销
时间/时区 bug 继续蔓延	高	统一 `date-utils`，禁止直接 `new Date('YYYY-MM-DD')`
导入中途失败造成数据破坏	高	事务 / 快照 / 预检 / 原子替换
提醒不稳定	中高	托盘常驻、开机启动、恢复补偿、提醒审计日志
代码继续漂移	中	建立 test/lint/format 脚本，冻结目录结构与模块边界

代码草案与测试清单

下面的代码不是“最终代码”，而是我建议你按这个方向落地的 **关键骨架**。

本地存储 schema。
当前仓库已经有 `tasks / reminders / pomodoro_sessions / settings / lists / tags / task_tags / saved_filters` 等表，但我建议你把“附件、AI 会话、排程快照、敏感设置”拆出来，而不是继续挤进 `tasks.attachments` 或普通 `settings`。现有 schema 和备份入口说明了这种拆分的必要性。 [42] [7]

```sql
-- 现有 tasks 保留，新增这些表
CREATE TABLE attachments (
  id TEXT PRIMARY KEY,
  task_id TEXT NOT NULL,
```

```sql
  original_name TEXT NOT NULL,
  stored_path TEXT NOT NULL,
  mime_type TEXT,
  size INTEGER,
  created_at TEXT NOT NULL
);

CREATE TABLE ai_threads (
  id TEXT PRIMARY KEY,
  title TEXT,
  provider TEXT NOT NULL,
  model TEXT NOT NULL,
  last_response_id TEXT,
  summary TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

CREATE TABLE ai_actions_log (
  id TEXT PRIMARY KEY,
  thread_id TEXT NOT NULL,
  action_type TEXT NOT NULL,  -- create_task / update_task / apply_schedule
  payload_json TEXT NOT NULL,
  applied INTEGER NOT NULL DEFAULT 0,
  created_at TEXT NOT NULL
);

CREATE TABLE plan_snapshots (
  id TEXT PRIMARY KEY,
  source TEXT NOT NULL,        -- agent / manual / import
  snapshot_json TEXT NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE schedule_constraints (
  date TEXT PRIMARY KEY,
  work_start TEXT,
  work_end TEXT,
  focus_block_min INTEGER DEFAULT 25,
  break_min INTEGER DEFAULT 5,
  no_schedule_json TEXT,       -- e.g. lunch, school pickup
  energy_level TEXT            -- low / normal / high
);

-- 注意：API key 不进入普通 settings，不默认进入 backup bundle
```

Agent 集成流程。
Responses API 适合你这种“本地工具执行 + 云模型推理”的模式：第一轮让模型决定要不要调本地工具，第二轮把 tool result 送回去拿自然语言解释。官方也明确说明了 function calling 和 chained responses 的标准流程。 [43]

```
// electron/services/ai-service.ts
export async function chatAndPlan(userText: string, threadId?: string) {
  const apiKey = loadSecret("openai_api_key");   // from safeStorage
  const context = await buildPlanningContext();  // tasks, due dates, free
slots, settings

  const first = await client.responses.create({
    model: "gpt-5.4-mini",
    previous_response_id: threadId ? await getLastResponseId(threadId) :
undefined,
    input: [
      { role: "system", content: SYSTEM_PROMPT },
      { role: "user", content: JSON.stringify({ userText, context }) }
    ],
    tools: [
      defineTool("create_tasks", createTasksSchema),
      defineTool("update_tasks", updateTasksSchema),
      defineTool("find_free_slots", freeSlotsSchema),
      defineTool("schedule_tasks", scheduleTasksSchema)
    ]
  });

  const toolOutputs = [];
  for (const item of first.output) {
    if (item.type !== "function_call") continue;

    const args = JSON.parse(item.arguments);
    const result = await runLocalTool(item.name, args, { dryRun: true });
    toolOutputs.push({
      type: "function_call_output",
      call_id: item.call_id,
      output: JSON.stringify(result)
    });
  }

  const final = toolOutputs.length
    ? await client.responses.create({
        model: "gpt-5.4-mini",
        previous_response_id: first.id,
        input: toolOutputs
      })
    : first;
```

```
  // 返回“草稿计划”，UI 负责确认；确认后再 apply
  return {
    responseId: final.id,
    assistantText: final.output_text,
    draftActions: extractDraftActions(toolOutputs)
  };
}
```

API key 存储 / 加密示例。
Electron 的 `safeStorage` 是当前桌面端最合适的默认方案；Linux 如果后端退化，就不要假装“已经安全”，要显式提示用户。 [40]

```
// electron/services/keyvault.ts
import { app, safeStorage } from "electron";
import fs from "node:fs";
import path from "node:path";

const secretFile = path.join(app.getPath("userData"), "secrets.json");

export function saveSecret(key: string, value: string) {
  if (!safeStorage.isEncryptionAvailable()) {
    throw new Error("OS encryption unavailable; use session-only mode or master
password fallback.");
  }

  const raw = fs.existsSync(secretFile)
    ? JSON.parse(fs.readFileSync(secretFile, "utf8"))
    : {};

  raw[key] = safeStorage.encryptString(value).toString("base64");
  fs.writeFileSync(secretFile, JSON.stringify(raw, null, 2), "utf8");
}

export function loadSecret(key: string): string | null {
  if (!fs.existsSync(secretFile)) return null;
  const raw = JSON.parse(fs.readFileSync(secretFile, "utf8"));
  if (!raw[key]) return null;
  return safeStorage.decryptString(Buffer.from(raw[key], "base64"));
}

export function getSecretHealth() {
  return {
    encryptionAvailable: safeStorage.isEncryptionAvailable(),
    backend:
      typeof safeStorage.getSelectedStorageBackend === "function"
        ? safeStorage.getSelectedStorageBackend()
```

```
        : "unknown"
  };
}
```

后台排程逻辑。
真正决定结果质量的，不是模型本身，而是你给模型喂的“约束”和本地排程器。建议 AI 只做 **解释、拆解、优先级建议**，排程则由本地确定性算法执行。 35

```
// electron/services/scheduling-engine.ts
export function draftDailySchedule(tasks, fixedBlocks, constraints) {
  const freeSlots = subtractFromWorkHours(constraints.workStart,
constraints.workEnd, fixedBlocks);

  const ranked = [...tasks]
    .filter(t => !t.completed && !t.scheduled)
    .sort((a, b) => {
      const dueScore = scoreDueDate(a) - scoreDueDate(b);
      const priorityScore = scorePriority(b) - scorePriority(a);
      const effortScore = scoreDuration(a) - scoreDuration(b);
      return dueScore || priorityScore || effortScore;
    });

  const plan = [];
  for (const task of ranked) {
    const needed = clamp(task.duration ?? 30, 15, 180);
    const slot = chooseBestSlot(freeSlots, needed, task, constraints);
    if (!slot) continue;

    plan.push({
      taskId: task.id,
      dueDate: slot.date,
      dueTime: slot.start,
      duration: slot.duration
    });

    consumeSlot(freeSlots, slot);
  }
  return plan; // draft only
}
```

Prompt 模板。
建议你至少准备三类模板：解析、排程、重排。并且全部要求“先出草稿，不直接提交”。

```
[系统提示词]
你是 plans-app 的本地排程助手。
```

```
目标不是聊天，而是把用户意图变成可确认的任务草稿或排程草稿。
你不能直接提交写操作；所有写操作都必须先返回 draft。
如果信息不足，先提出最少量澄清问题。
优先遵守：已有固定日程 > 截止时间 > 用户指定时段 > 优先级 > 任务时长。

[解析模板]
把下面文本解析成任务列表，输出 JSON：
- title
- dueDate
- dueTime
- duration
- priority
- recurrenceRule
- ambiguity

[排程模板]
根据给定任务、固定时间块、工作时段，输出“今日草稿计划”：
- 不要覆盖 fixedBlocks
- 尽量把高优先级且临近截止的任务前置
- 每个时间块 25~90 分钟
- 输出 explain 字段说明原因

[重排模板]
把未完成任务重排到本周剩余时间。
保留已完成和已锁定时间块。
如果任务太多，明确指出超载而不是强行排满。
```

建议测试。
当前仓库文档多次提到测试，但工程入口还没立起来；我建议至少先把下面这些补齐。 [31] [28] [30]

- **单元测试**
- 日期工具：`today`、`next 7 days`、`YYYY-MM-DD` 解析、跨午夜。
- 重复规则：每周一三五、每月最后一天、count/endDate/exception。
- 排程引擎：固定块不被覆盖、过载时能诚实返回“排不下”。
- KeyVault：保存/读取/无加密后端提示。

- AI 草稿提取：非法 JSON、空 tool result、重复 call_id。

- **集成测试**

- `task:create/update/delete/list/search` 与提醒联动。
- 导入 bundle 的预检、失败回滚、附件恢复。
- 关闭主窗、托盘常驻、开机启动、睡眠恢复。

- agent 的 draft-confirm-apply-undo 全链路。

- **E2E / QA 检查清单**

- 输入“明天下午 3 点给客户回电话 20 分钟”是否直接出正确草稿。
- 对 agent 说“把今天没做完的全部重排到本周”是否先展示草稿而不是直接改库。
- 撤销按钮是否能恢复到 agent 应用前状态。
- 导出的 bundle 在另一台机器导入后，附件是否还能打开。
- 关掉主窗口后，提醒是否继续可用。
- 睡眠 30 分钟再恢复，番茄钟和提醒是否校正。
- 断网时 agent 是否优雅降级，而不是让页面悬死。
- key 是否不会出现在普通设置页、日志和默认备份中。

---

9 App.tsx
https://github.com/Distance0x/plans-app/blob/main/src/App.tsx

2 4 README.md
https://github.com/Distance0x/plans-app/blob/main/README.md

3 10-功能缺口与迭代规划.md
https://github.com/Distance0x/plans-app/blob/main/docs/10-功能缺口与迭代规划.md

5 34 https://platform.openai.com/docs/api-reference/responses/retrieve
https://platform.openai.com/docs/api-reference/responses/retrieve

6 db.ts
https://github.com/Distance0x/plans-app/blob/main/electron/database/db.ts

7 backup-handler.ts
https://github.com/Distance0x/plans-app/blob/main/electron/ipc/backup-handler.ts

8 timer-handler.ts
https://github.com/Distance0x/plans-app/blob/main/electron/ipc/timer-handler.ts

10 TaskFilters.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/tasks/TaskFilters.tsx

11 14 https://www.electronjs.org/docs/latest/tutorial/context-isolation
https://www.electronjs.org/docs/latest/tutorial/context-isolation

12 index.ts
https://github.com/Distance0x/plans-app/blob/main/electron/main/index.ts

13 docs/03-数据库设计.md
https://github.com/Distance0x/plans-app/blob/92665eb9b9dc5945444f1fd28a30e924f359c7c4/docs/03-%E6%95%B0%E6%8D%AE%E5%BA%93%E8%AE%BE%E8%AE%A1.md

15 38 https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API
https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API

16 Calendar.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/calendar/Calendar.tsx

17 FloatingApp.tsx
https://github.com/Distance0x/plans-app/blob/main/src/FloatingApp.tsx

18 powerMonitor | Electron
https://www.electronjs.org/docs/latest/api/power-monitor/?utm_source=chatgpt.com

19 reminder-engine.ts
https://github.com/Distance0x/plans-app/blob/main/electron/services/reminder-engine.ts

20 tray.ts
https://github.com/Distance0x/plans-app/blob/main/electron/utils/tray.ts

21 TaskList.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/tasks/TaskList.tsx

22 task-handler.ts
https://github.com/Distance0x/plans-app/blob/main/electron/ipc/task-handler.ts

23 QuickAddTask.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/tasks/QuickAddTask.tsx

24 TaskForm.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/tasks/TaskForm.tsx

25 src/components/tasks/TaskForm.tsx
https://github.com/Distance0x/plans-app/blob/92665eb9b9dc5945444f1fd28a30e924f359c7c4/src/components/tasks/TaskForm.tsx

26 docs/04-实现计划.md
https://github.com/Distance0x/plans-app/blob/92665eb9b9dc5945444f1fd28a30e924f359c7c4/docs/04-%E5%AE%9E%E7%8E%B0%E8%AE%A1%E5%88%92.md

27 TaskItem.tsx
https://github.com/Distance0x/plans-app/blob/main/src/components/tasks/TaskItem.tsx

28 04-实现计划.md
https://github.com/Distance0x/plans-app/blob/main/docs/04-实现计划.md

29 electron.d.ts
https://github.com/Distance0x/plans-app/blob/main/src/types/electron.d.ts

31 02-技术架构设计.md
https://github.com/Distance0x/plans-app/blob/main/docs/02-技术架构设计.md

32 main.tsx
https://github.com/Distance0x/plans-app/blob/main/src/main.tsx

33 https://ticktick.com/home
https://ticktick.com/home

35 https://www.sunsama.com/features/timeboxing
https://www.sunsama.com/features/timeboxing

36 https://www.electronjs.org/docs/latest/api/tray/
https://www.electronjs.org/docs/latest/api/tray/

37 https://icalendar.org/RFC-Specifications/iCalendar-RFC-5545/
https://icalendar.org/RFC-Specifications/iCalendar-RFC-5545/

39 https://platform.openai.com/docs/guides/conversation-state
https://platform.openai.com/docs/guides/conversation-state

40 https://www.electronjs.org/docs/latest/api/safe-storage
https://www.electronjs.org/docs/latest/api/safe-storage

41 https://openai.com/api/pricing/
https://openai.com/api/pricing/

42 electron/database/db.ts
https://github.com/Distance0x/plans-app/blob/92665eb9b9dc5945444f1fd28a30e924f359c7c4/electron/database/db.ts

43 https://platform.openai.com/docs/guides/function-calling/how-do-i-ensure-the-model-calls-the-correct-function
https://platform.openai.com/docs/guides/function-calling/how-do-i-ensure-the-model-calls-the-correct-function